Parallel Programming

Shared memory and locks, race conditions and guidelines

What we have seen last week:

Have been studying **parallel algorithms** using fork-join

- Lower span via parallel tasks

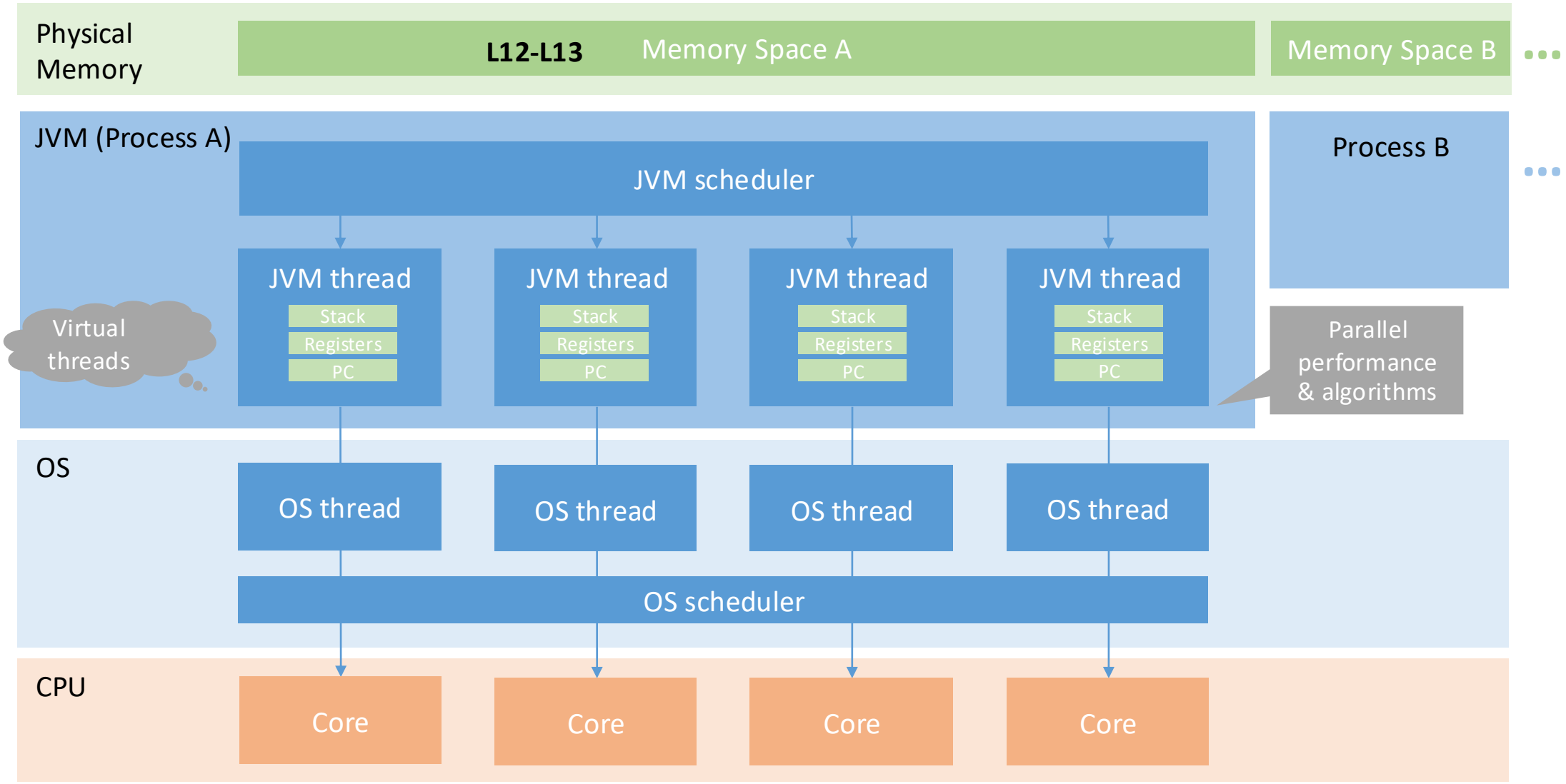
Algorithms all had a simple structure to avoid race conditions

- Each thread had memory “only it accessed”, e.g: array sub-range
- On **`fork`**, “loan” some memory to “forkee” and do not access that memory again until after **`join`** on the “forkee”

Strategy won’t work well when:

- Memory accessed by threads is overlapping or unpredictable
- Threads are doing independent tasks needing access to same resources (rather than implementing the same algorithm)

Big picture (part I)



Topics Today

Shared memory and the concept of locks

Race conditions: Data races and bad interleavings

Guidelines for concurrent programming

Shared memory and the concept of locks

Managing state

Immutability

- Data do not change
- Best option, should be used when possible

Isolated mutability

- Data can change, but only one thread/task can access them

Mutable/shared data

- Data can change, all tasks/threads can potentially access them
  → protect state by allowing only one task / thread to access it at a time

Canonical example

Correct code in a single-threaded world

```
class BankAccount {
  private int balance = 0;
  int  getBalance()       { return balance; }
  void setBalance(int x) { balance = x; }
  void withdraw(int amount) {
    int b = getBalance();
    if(amount > b)
      throw new WithdrawTooLargeException();
    setBalance(b – amount);
  }
  … // other operations like deposit, etc.
}
```

Race condition

Interleaved **`withdraw(100)`** calls on the same account

- Assume initial **`balance == 150`**

Thread 1

```
int b = getBalance();




if(amount > b)
  throw new …;
setBalance(b – amount);
```

Thread 2

```
int b = getBalance();
if(amount > b)
  throw new …;
setBalance(b – amount);
```

Time

“Lost withdraw”!

Incorrect “fix”

It is tempting and almost always wrong to fix a bad interleaving by rearranging or repeating operations, such as:

```
void withdraw(int amount) {
  if(amount > getBalance())
    throw new WithdrawTooLargeException();
  // maybe balance changed
  setBalance(getBalance() – amount);
}
```

This fixes nothing!

- Narrows the problem by one statement
- (Not even that since the compiler could turn it back into the old version because you didn’t indicate need to synchronize)
- And now a negative balance is possible – why?

Mutual exclusion

Sane fix: Allow at most one thread to withdraw from account **A** at a time

- Exclude other simultaneous operations on **A** too (e.g., deposit)

Called **mutual exclusion**: One thread using a resource (here: an account) means another thread must wait

- a.k.a. **critical sections**, which technically have other requirements

Programmer must implement critical sections

- "The compiler" has no idea what interleavings should or should not be allowed in your program
- But you need language primitives to do it!

Critical sections and mutual exclusion

Critical section

Piece of code that may be executed by at most one process (thread) at a time

Mutual exclusion

Algorithm to implement a critical section

```
acquire_mutex();        // entry algorithm
...                     // critical section
release_mutex();        // exit algorithm
```

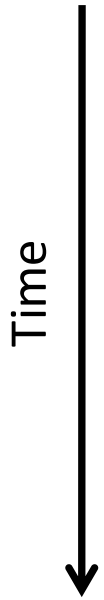
Wrong!

Why can't we implement our own mutual-exclusion protocol?

- It is theoretically possible to implement a custom mutual-exclusion protocol under certain assumptions, but real-world architectures and language memory models make it highly unreliable and impractical without hardware support

```
class BankAccount {
  private int balance = 0;
  private boolean busy = false;
  void withdraw(int amount) {
    while(busy) { /* "spin-wait" */ }
    busy = true;
    int b = getBalance();
    if(amount > b)
      throw new WithdrawTooLargeException();
    setBalance(b – amount);
    busy = false;
  }
  // deposit would spin on same boolean
}
```

Just moved the problem!



Thread 1

```
while(busy) { }

busy = true;

int b = getBalance();




if(amount > b)
  throw new …;
setBalance(b – amount);
```

Thread 2

```
while(busy) { }

busy = true;

int b = getBalance();
if(amount > b)
  throw new …;
setBalance(b – amount);
```

"Lost withdraw"!

What we need

We need help from the language: **Locks**

A primitive with atomic operations **`new`**, **`acquire`**, **`release`**

- **`new`**: make a new lock, initially “not held”
- **`acquire`**: blocks if this lock is already currently “held”
  - Once “not held”, makes lock “held” [all at once!]
- **`release`**: makes this lock “not held”
  - If >= 1 threads are blocked on it, exactly 1 will acquire it

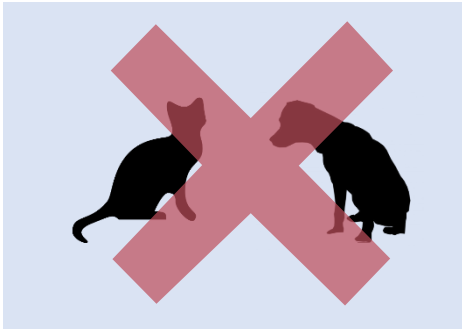
How can this be implemented?

- Need to “check if held and if not make held” “all-at-once”
- Uses special hardware and O/S support
- Here, we take this as a primitive and use it

Required properties of mutual exclusion

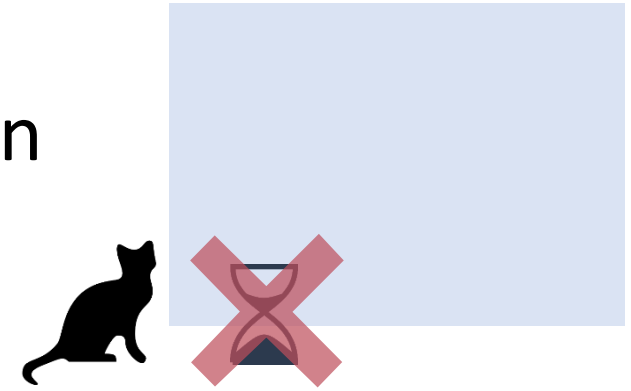
Safety Property

At most one process executes the critical section code



Liveness

Minimally: acquire_mutex must terminate in finite time when no process executes in the critical section



Now some Java: Synchronized block (recap)

Java has built-in support for locks

- Uses internal lock of an object (intrinsic lock)
- **`synchronized`** statement
- Re-entrant

```
synchronized (object)
{
   statements
}
```

1. Every object “is a lock” in Java
2. Acquires the lock, blocking if necessary
   If you get past the {, you have the lock
3. Releases the lock “at the matching }

External locks

Java also offers external locks (e.g. in package `java.util.concurrent.locks`)

- Less easy to use
- But support more sophisticated locking idioms, e.g. for reader-writer scenarios

```
private Lock lk = new ReentrantLock();

lk.lock();   // Blocks until the lock is acquired

...          // Critical section

lk.unlock(); // Releases the lock
```

Not correct yet

Almost-correct pseudocode

```
class BankAccount {
   private int balance = 0;
   private Lock lk = new ReentrantLock();
   …
   void withdraw(int amount) {
      lk.lock(); // May block
      int b = getBalance();
      if(amount > b)
         throw new WithdrawTooLargeException();
      setBalance(b – amount);
      lk.unlock();
   }
   // deposit would also acquire/release lk
}
```

One lock for each account, initially not held

entering CS

What if an exception occurs?

leaving CS

Without try-finally, if an exception occurs before unlock(), the lock is never released

Exceptions and locks

```
class BankAccount {
   private int balance = 0;
   private Lock lk = new ReentrantLock();
   …
   void withdraw(int amount) {
      lk.lock(); // May block
      try {
         int b = getBalance();
         if(amount > b)
            throw new WithdrawTooLargeException();
         setBalance(b – amount);
      } finally {
         lk.unlock();
      }
   }
}
```

One lock for each account, initially not held

entering CS

leaving CS

If an exception (like WithdrawTooLargeException) occurs, the lock will still be released

Other possible issues

Incorrect: Use different locks for **`withdraw`** and **`deposit`**

- Mutual exclusion works only when using same lock
  `balance` field is the shared resource being protected

Poor performance: Use same lock for every bank account

- No simultaneous operations on different accounts
- Each account should get its own lock

Waiting for lock

lock()

- Acquires the lock and **blocks indefinitely** until it is available

tryLock()

- Attempts to acquire the lock **without blocking**
- Returns true if successful, false if the lock is already held
- Supports timeouts

```
if (lk.tryLock(100, TimeUnit.MILLISECONDS)) { // Wait up to 100ms
    try {
        // Critical section
    } finally {
        lk.unlock();
    }
} else {
    // Lock not acquired, handle accordingly
}
```

Avoiding long waits or deadlocks, implementing fallback strategies

Code examples: PP-L12-01ReentrantLock, PP-L12-02TryLock

(additional material)

Code example using tryLock()

```
public class MyThread implements Runnable {

   private Lock lock;

   public MyThread(Lock lock) {
      this.lock = lock;
   }

   public void run() {
      if(lock.tryLock(2000, TimeUnit.MILLISECONDS)) {
         try { printMessage("Lock acquired"); }
         finally {
            printMessage("Lock released");
            lock.unlock();
         }
      }
      else {
         printMessage("Lock not acquired - return");
         return;
      }
   }
}
```

Why reentrancy

If **`withdraw`** and **`deposit`** use the same lock, then simultaneous calls to these methods are properly synchronized

But what about **`getBalance`** and **`setBalance`**?

Assume they are **`public`**, which may be reasonable

- If they **`do not`** acquire the same lock, then a race between **`setBalance`** and **`withdraw`** could produce a wrong result
- If they **`do`** acquire the same lock, then **`withdraw`** would block forever because it tries to acquire a lock it already has

```
public void setBalance(int x) { .. }


public int getBalance() { .. }


public void withdraw(int amount) {
    ..
    b = getBalance()
    ..
    setBalance(b – amount);
    ..
}
public void deposit(int amount){
    ..
    b = getBalance()
    ..
    setBalance(b + amount);
    ..
}
```

Re-acquiring locks?

One approach:

Can't let outside world call **`setBalance1`**

Can't have **`withdraw`** call **`setBalance2`**

Another approach:

Can modify the meaning of the Lock to support re-entrant locks

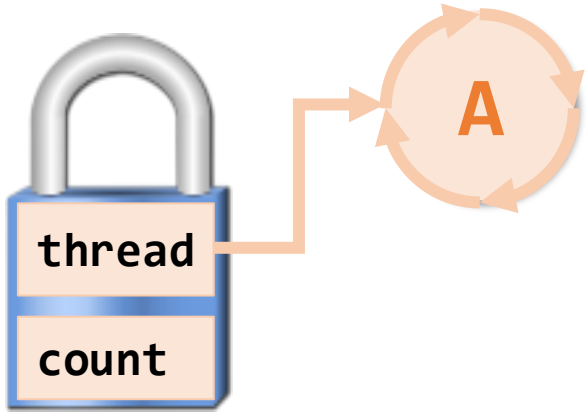
- Java does this
- Then just use a single **`setBalance`**

```
int setBalance1(int x) {
  balance = x;
}

int setBalance2(int x) {
  lk.lock();
  setBalance1(x);
  lk.unlock();
}

void withdraw(int amount) {
  lk.lock();
  …
  setBalance1(b – amount);
  lk.unlock();
}
```

Reentrant lock



A **reentrant lock** (a.k.a. **recursive lock**)
"remembers"

- The thread (if any) that currently holds it
- A count

When the lock goes from not-held to held, the count is set to 0

If (code running in) the current holder calls **`lock(acquire)`**:

- It does not block
- It increments the count

On **`unlock(release)`**:

- If the count is > 0, the count is decremented
- If the count is 0, the lock becomes not-held

Re-entrant locks work

This simple code works fine provided **`lk`** is a reentrant lock

Okay to call **`setBalance`** directly

Okay to call **`withdraw`** (won't block forever)

```
int setBalance(int x) {
  lk.lock();
  balance = x;
  lk.unlock();
}

void withdraw(int amount) {
  lk.lock();
  …
  setBalance(b – amount);
  lk.unlock();
}
```

synchronized versus Lock API

Aspect	synchronized	Lock API (java.util.concurrent.locks)
Lock handling	Automatic (JVM managed)	Manual (lock() / unlock())
Lock release	Automatic	Must be done explicitly (finally!)
Scope	Method or block scoped	Flexible (can span multiple methods)
Waiting behavior	Thread blocks indefinitely	tryLock() avoids blocking
Interruptibility	Not interruptible while waiting	lockInterruptibly() supports interrupt
Flexibility	Low	High
Complexity	Simple, less error-prone	More complex, error-prone
Deadlock risk	Lower	Higher (if unlock forgotten)
Typical use	Simple synchronization	Advanced concurrency control

Rule of thumb:

- Use synchronized → default choice
- Use Lock → only if you need advanced features
- Prefer concurrent collections when possible → they handle synchronization internally (e.g., ConcurrentHashMap, BlockingQueue)

Race conditions: Data races and bad interleavings

Low-level and high-level race conditions

A **Race Condition** occurs in concurrent programming when the correctness of the system depends on the specific interleaving or ordering of operations executed by multiple threads or processes.

- Typically, problem is some intermediate state that “messes up” a concurrent thread that “sees” that state

Not all race conditions are the same - some are **always errors (low-level, data races),** while others **depend on the algorithm** and may only be problematic under certain conditions **(high-level, bad interleaving)**

This lecture makes a distinction between data races and bad interleavings, both **instances of race condition bugs**

- In both cases we have interleavings that impact program correctness

The distinction

Data Race [aka low-level race condition, low semantic level]

Erroneous program behavior caused by insufficiently synchronized accesses of a shared resource by multiple threads, e.g. Simultaneous read/write or write/write of the same memory location

- Occurs at memory / CPU level, caused by unsynchronized memory access
- Always an error
- Example: bank account balance, shared counter

Bad Interleaving [aka high-level race condition, high semantic level]

Erroneous program behavior caused by an unfavorable execution order of a multithreaded algorithm that makes use of otherwise well synchronized resources

- Algorithmic level, caused by unfavorable execution order
- “Bad” depends on your specification
- Example: bank transfers executing out of order

Low-level race condition

```
public void setBalance(int x) { ..  }

public int getBalance() { ..  }

public void withdraw(int amount) {
    ..
    b = getBalance();
    ..
    setBalance(b – amount);
    ..
}
public void deposit(int amount){
    ..
    b = getBalance();
    ..
    setBalance(b + amount);
    ..
}
```

Multiple threads access balance concurrently

Read / write without synchronization

-> Data race (low-level): setBalance is not synchronized, threads can write balance without synchronization

High-level race condition

```
public synchronized void setBalance(int x) {
..   }

public synchronized int getBalance() { ..   }

public void withdraw(int amount) {
    ..
    b = getBalance(); // synchronized
    ..
    setBalance(b – amount); // synchronized
    ..
}
public synchronized void deposit(int amount){
    ..
    b = getBalance();
    ..
    setBalance(b + amount);
    ..
}
```

Synchronized methods, but withdraw not atomic

-> *No* data race: All access to balance are synchronized -> no concurrent r/w or w/w

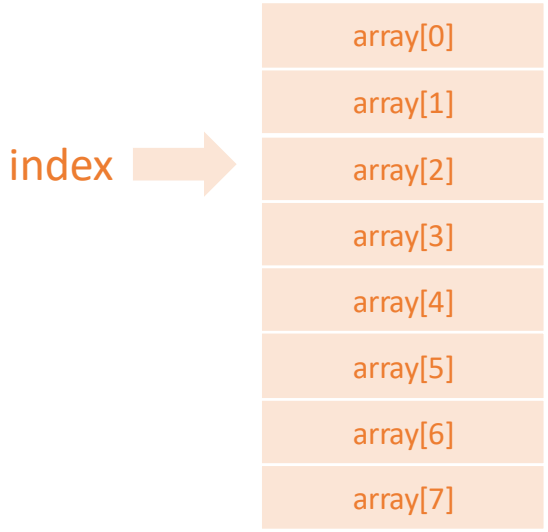
-> Bad interleaving (high-level): Execution order of operations (get -> check -> set) in an unfavorable way leading to incorrect balance (“lost withdraw”)

Example: Bounded Stack

```
public class Stack <E> {
E[] array;
int index;

    public Stack(int entries){
        // hack to generate a generic array,
        // initialized with NIL values
        array = (E[]) new Object[entries];
        index = 0;
    }
…

}
```



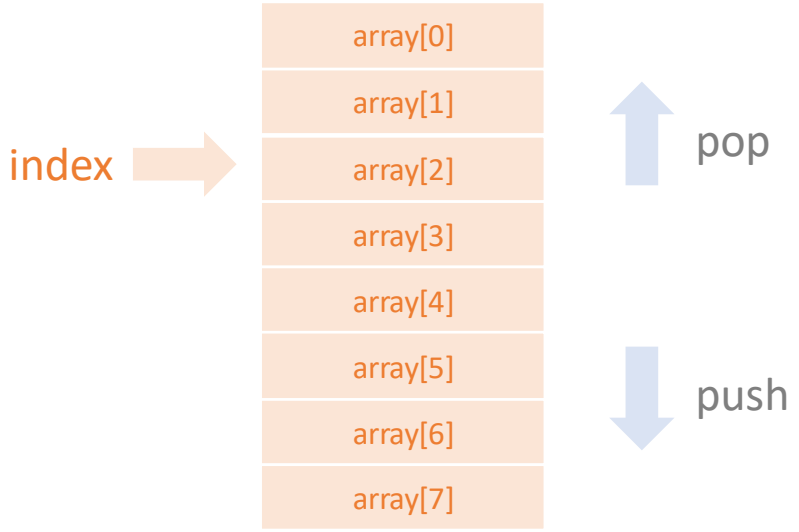
Example: Bounded Stack

```
public class Stack <E> {
…
    synchronized boolean isEmpty() {
        return index==0;
    }

    synchronized void push(E val) throws StackFullException {
        if (index==array.length)
            throw new StackFullException();
        array[index++] = val;
    }

    synchronized E pop() throws StackEmptyException {
        if (index==0) throw new StackEmptyException();
        return array[--index];
    }
}
```

Can cause race conditions **even if it only reads**, because it might see inconsistent state due to concurrent modifications



Peek helper method

```
public class Stack <E> {
…
    E peek() {
        E ans = pop();
        push(ans);
        return ans;
    }
}
```

wrong

`peek`, concurrently speaking

`peek` does not modify shared data

- Push and pop are synchronized, ensuring no data races on the underlying structure
- It is a “reader” not a “writer”
- Every access to shared data is protected

But still a problem: `peek` introduces an inconsistent intermediate state

- Bad interleavings where it sees a partially modified structure
- Violation of stack invariants
- Incorrect behavior in algorithms that rely on a consistent view of the stack

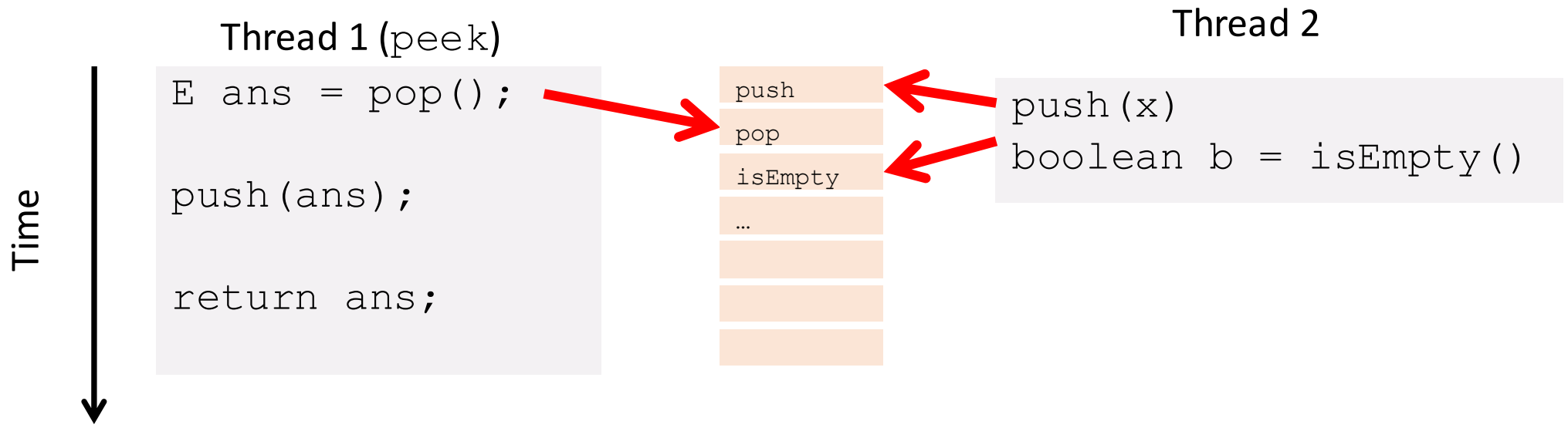
Even though each operation is thread-safe, the system as a whole is **not behaviorally safe**

Takeaway: The absence of data races does **not** mean a program is free from race-condition bugs - exposing inconsistent intermediate states can violate key invariants and lead to incorrect behavior

peek and isEmpty

Property we want (invariant): If there has been a **`push`** and no **`pop`**, then **`isEmpty`** returns **`false`**

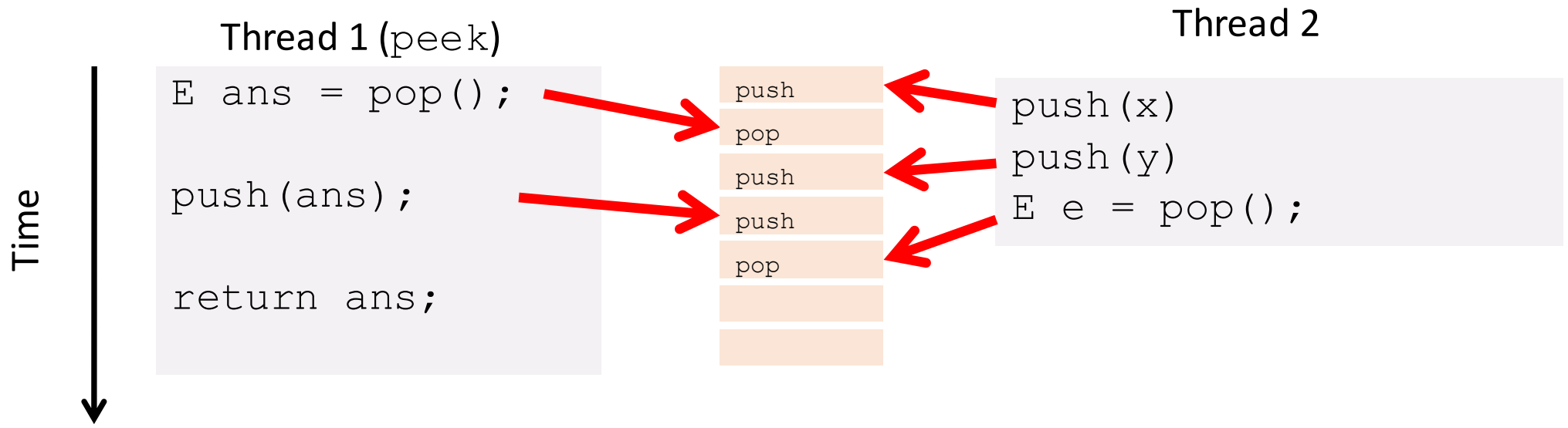
With **`peek`** as written, property can be violated



peek and LIFO

Property we want: Values are returned from **`pop`** in LIFO order

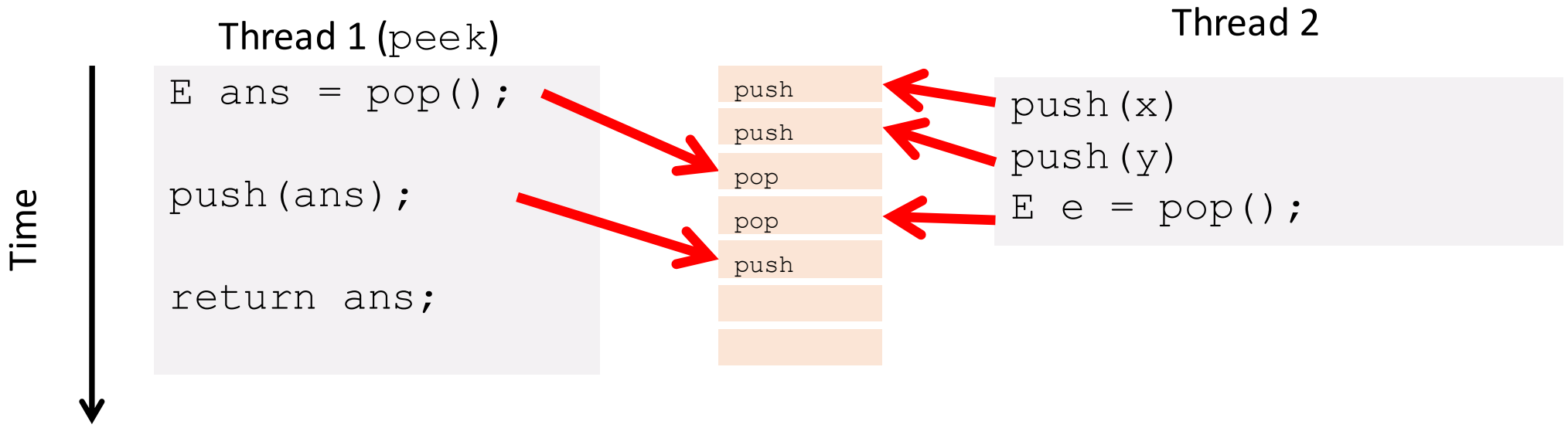
With **`peek`** as written, property can be violated



peek and LIFO

Property we want: Values are returned from **`pop`** in LIFO order

With **`peek`** as written, property can be violated



The fix

In short, **`peek`** needs synchronization to disallow interleavings

- The key is to make a larger critical section
- Re-entrant locks allow calls to **`push`** and **`pop`**

```
class Stack<E> {
  …
  synchronized E peek(){
     E ans = pop();
     push(ans);
     return ans;
  }
}
```

Getting it right

Avoiding race conditions on shared resources is difficult

- Decades of bugs have led to some conventional wisdom: general techniques that are known to work

Rest of lecture distills key ideas and trade-offs

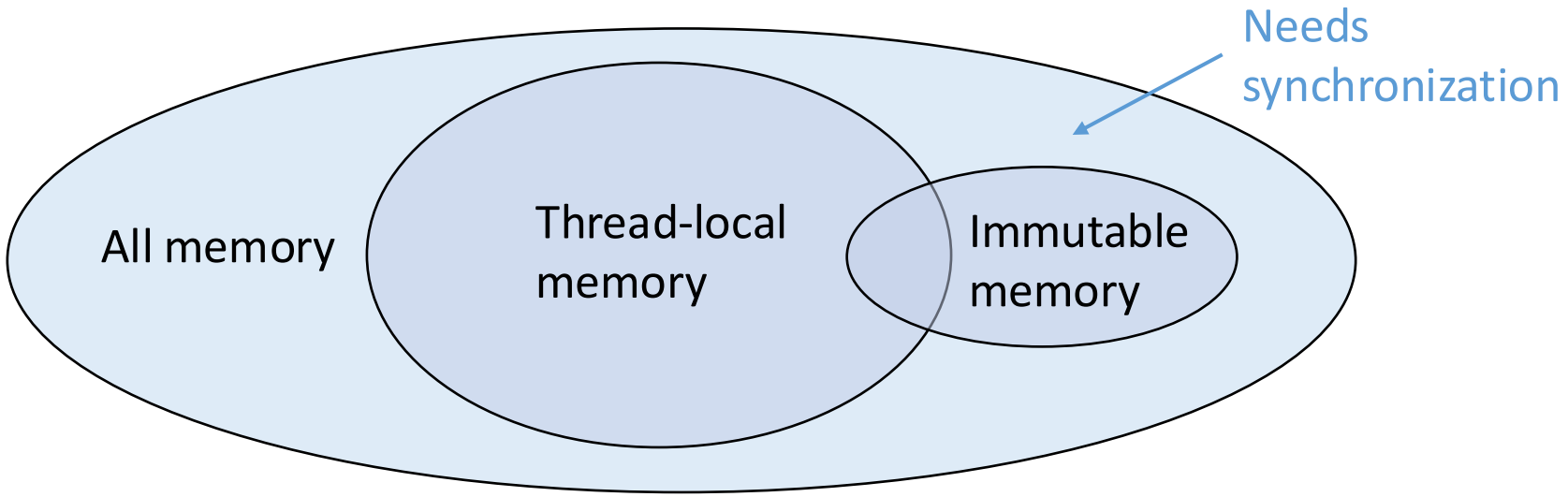
- Parts paraphrased from “Java Concurrency in Practice”
- But none of this is specific to Java or a particular book!

Guidelines for concurrent programming

3 choices

For every **memory location** (e.g., object field) in your program, you must obey at least one of the following:

1. **Thread-local:** Do not use the location in > 1 thread
2. **Immutable:** Do not write to the memory location
3. **Synchronized:** Use synchronization to control access to the location



Thread-local

Whenever possible, do not share resources

Easier to have each thread have its own **thread-local copy** of a resource than to have one with shared updates

- This is correct only if threads do not need to communicate through the resource
- Note: Because each call-stack is thread-local, never need to synchronize on local variables

In typical concurrent programs, the vast majority of objects should be thread-local: shared-memory should be rare – minimize it

Immutable

Whenever possible, do not update objects

- Make new objects instead

If a location is only read, never written, then no synchronization is necessary!

- Simultaneous reads are not races and not a problem

In practice, programmers usually over-use mutation – minimize it

The rest

After minimizing the amount of memory that is (1) thread-shared and (2) mutable, we need guidelines for how to use locks to keep other data consistent

Guideline #0: No data races

Never allow two threads to read/write or write/write the same location at the same time. Do not make any assumptions on the orders of reads or writes.

Necessary: In Java or C, a program with a data race is always wrong

Not sufficient: Our `peek` example had no data races

Consistent locking

Guideline #1: For each location needing synchronization, have a lock that is always held when reading or writing the location

We say the lock **guards** the location

- The same lock can (and often should) guard multiple locations
- Clearly document the guard for each location

In Java, often the guard is the object containing the location

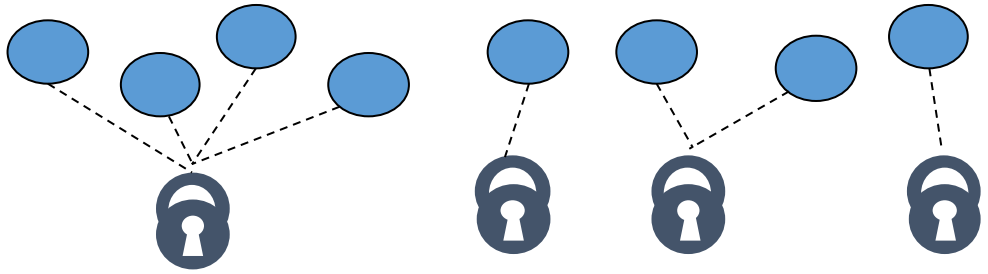
- `this` inside the object's methods
- But also often guard a larger structure with one lock to ensure mutual exclusion on the structure

Consistent locking continued

The mapping from locations to guarding locks is conceptual

- Up to you as the programmer to follow it

It partitions the shared-and-mutable locations into "which lock"



Consistent locking is **not sufficient:** It prevents all data races but still allows bad interleavings. Our `peek` example used consistent locking

Beyond consistent locking

Consistent locking is an excellent guideline

- A “default assumption” about program design

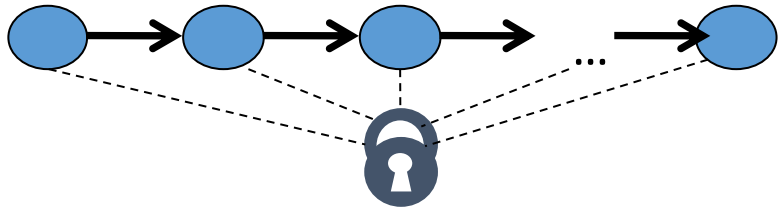
Consistent locking is not required for correctness: Can have different program phases that use different invariants

- Provided all threads coordinate moving to the next phase

Lock granularity

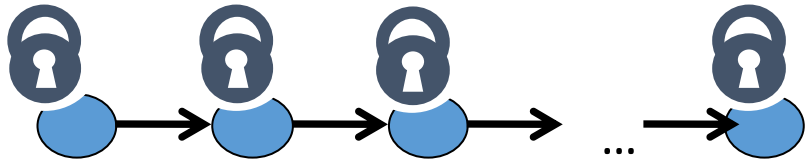
Coarse-grained: Fewer locks, i.e., more objects per lock

- Example: One lock for entire data structure (e.g., array)
- Example: One lock for all bank accounts



Fine-grained: More locks, i.e., fewer objects per lock

- Example: One lock per data element (e.g., array index)
- Example: One lock per bank account
-



"Coarse-grained vs. fine-grained" is really a continuum

Trade-offs

Coarse-grained advantages

- Simpler to implement
- Faster/easier to implement operations that access multiple locations (because all guarded by the same lock)
- Much easier: operations that modify data-structure shape

Fine-grained advantages

- More simultaneous access (performance when coarse-grained would lead to unnecessary blocking)

Guideline #2: Start with coarse-grained (simpler) and move to fine-grained (performance) only if contention on the coarser locks becomes an issue. Alas, often leads to bugs.

Critical-section granularity

A second, orthogonal granularity issue is critical-section size

- How much work to do while holding lock(s)

If critical sections run for too long:

- Performance loss because other threads are blocked

If critical sections are too short:

- Bugs because you broke up something where other threads should not be able to see intermediate state
- Performance loss because of frequent thread switching and cache trashing

Guideline #3: Do not do expensive computations or I/O in critical sections, but also don't introduce race conditions

Example

Suppose we want to change the value for a key in a hashtable without removing it from the table

Assume **`lock`** guards the whole table

Critical section was **too long**

(table locked during expensive call)

```
synchronized(lock) {
  v1 = table.lookup(k);
  v2 = expensive(v1);
  table.remove(k);
  table.insert(k,v2);
}
```

Example

Suppose we want to change the value for a key in a hashtable without removing it from the table

Assume **`lock`** guards the whole table

Critical section was **too short**

(if another thread updated the entry, we will lose an update)

```
synchronized(lock) {
  v1 = table.lookup(k);
}
v2 = expensive(v1);
synchronized(lock) {
  table.remove(k);
  table.insert(k,v2);
}
```

Example

Suppose we want to change the value for a key in a hashtable without removing it from the table

Assume **`lock`** guards the whole table

Critical section was **just right**

(if another update occurred, try our update again)

```
done = false;
while(!done){
    synchronized(lock) {
        v1 = table.lookup(k);
    }
    v2 = expensive(v1);
    synchronized(lock) {
        if(table.lookup(k)==v1) {
            done = true;
            table.remove(k);
            table.insert(k,v2);
        }
    }
}
```

Atomicity

An operation is **atomic** if no other thread can see it partly executed

- Atomic as in "appears indivisible"
- Typically want Abstract Data Type (ADT) operations atomic, even to other threads running operations on the same ADT

Guideline #4: Think in terms of what operations need to be atomic

- Make critical sections just long enough to preserve atomicity
- Then design the locking protocol to implement the critical sections correctly

That is: Think about atomicity first and locks second

Don’t roll your own

It is rare that you should write your own data structure

Provided in standard libraries

Particularly true for concurrent data structures

Standard **thread-safe** libraries like **`ConcurrentHashMap`** written by world experts

Practical Guideline: Use built-in libraries whenever they meet your needs

Guideline for this course: do everything to understand it yourself!